UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS ECONÓMICO ADMINISTRATIVAS

MAESTRÍA EN CIENCIA DE LOS DATOS

Programación I

Reporte Prectica 1.

P R E S E N T A

Juan Manuel Seimandi Gollaz

12/10/2026

Zapopan, Jalisco

CUCEA
El mejor lugar para el talento

MCD MAESTRÍA EN CIENCIA DE LOS DATOS

1° Procedimiento.

El depurador transforma solicitudes de soporte en registros normalizados sin modificar la carpeta datos. El resultado fue de 24 registros válidos y 6 rechazados sobre 30 líneas de datos, incluida la vacía. La limpieza permitió reunir variantes de escritura en tres categorías.

Partimos de actividad_inicio.py y conservamos su lógica de funciones, lectura por líneas y escritura de resultados. Se ajusto la revisión de duplicados para respetar el orden del contrato: línea vacía, cantidad de campos, campo vacío, ID, correo, categoría y duplicado. Si una solicitud repite un ID y además tiene un correo incorrecto, se registra “Correo inválido”. Solo se agrega el ID al conjunto después de aceptar todos los campos.

En el ID se eliminan espacios extremos y se convierten letras a mayúsculas; se exigen SOL- y cuatro dígitos ASCII. El correo se convierte a minúsculas y se verifica una sola @, partes no vacías, ausencia de espacios y un punto interior en el dominio. En categorías se unifican espacios, guiones y guiones bajos, conservando tildes. La descripción conserva mayúsculas y tildes, pero reduce los espacios consecutivos a uno.

El diagnóstico de “ mx-gdl-0042 ” produjo longitud 15, slices “mx” y “0042”, normalización “MX-GDL-0042” y find("xyz") igual a -1. Las cadenas son inmutables: strip() no cambia la variable original si no se reasigna. in devuelve un booleano; find() devuelve una posición o -1. Usar ese -1 como índice recuperaría el último carácter, aunque la búsqueda hubiera fallado. capitalize() transforma SQL en Sql y puede alterar una sigla.

Las rutas se resuelven desde el directorio del script. Primero se abre la entrada en modo r y se comprueba la cabecera exacta; después se abren los resultados en w. Los rechazos conservan línea física, primer motivo y texto original sin el salto final. El resumen se escribe con json.dump y se reabre con json.load; cada rechazo se interpreta con json.loads. También se cuentan las filas limpias sin cabecera. Todas las aperturas de texto usan with y UTF-8.

Los controles calculados fueron 30 = 24 + 6 y 9 + 9 + 6 = 24. Acceso y pago_en_línea representan 37.5 % cada uno de los aceptados; soporte_técnico representa 25 %. Los rechazos corresponden a las líneas físicas 26 a 31: vacía, correo inválido, ID inválido, campo vacío, cantidad de campos incorrecta e ID duplicado. Las tasas son 6/30 = 20 % de rechazo y 24/30 = 80 % de aceptación.

2. Evidencias.

Se realizaron validaciones sobre el procesamiento del código para evaluar la respuesta del código cuando un dato no facilita el acceso o la lectura,

2.1. Dos Ejecuciones consecutivas.

Esperado: conservar los conteos de solicitudes_limpias.txt, rechazos.jsonl y resumen.json, y añadir una línea al historial por cada ejecución exitosa.

Observado: ambas ejecuciones produjeron 30 totales, 24 válidos y 6 rechazos, con categorías 9, 9 y 6. El historial pasó de 0 a 1 y después a 2 líneas. El modo w reemplaza los datos; el modo a agrega la nueva ejecución.

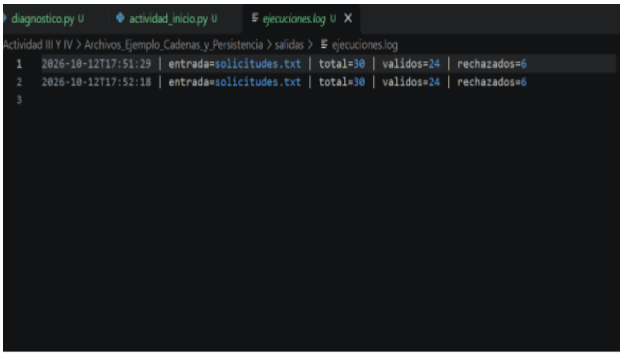


Figura 1. Ejecuciones

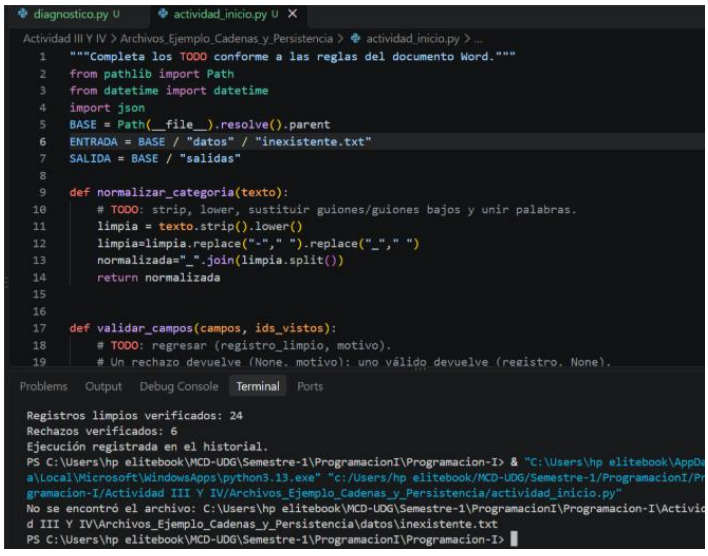


Figura 2. Conteo por categoría

2.2 Ruta inexistente.

Esperado: informar qué archivo no se encontró sin reemplazar resultados ni añadir una ejecución exitosa.

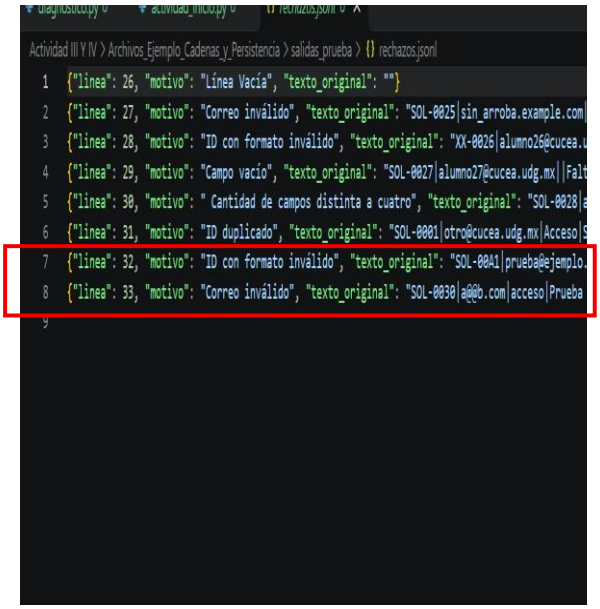
Observado: se mostró “No se encontró el archivo” seguido de la ruta datos/inexistente.txt. El bloque except FileNotFoundError capturó ese fallo.



Prueba 3. Ruta Inexistente

2.3 Registros incorrectos.

Esperado: al añadir SOL-00A1 y a@@b.com en registros distintos, aumentar el total a 32 y los rechazos a 8, conservando 24 válidos. Observado: SOL-00A1 se rechazó en la línea física 32 por ID con formato inválido; a@@b.com, en la línea 33 por correo inválido. Las categorías permanecieron en 9, 9 y 6. La tasa de rechazo de esta copia es 8/32 = 25 %, diferente del 20 % de la entrada original. La captura de resumen.json proporcionada coincide con estos conteos.



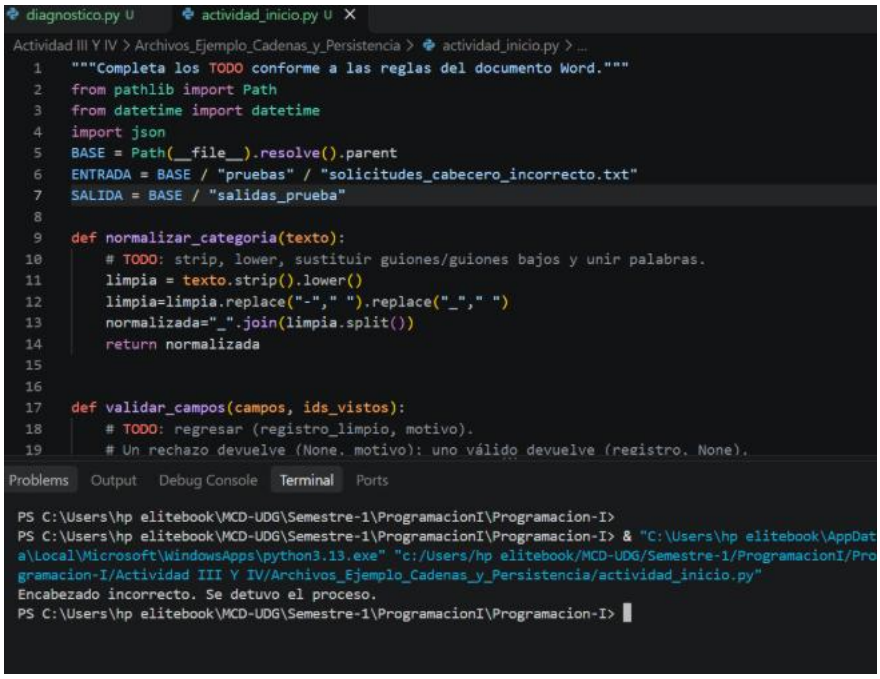
Prueba 4. Log rechazos



Prueba 5. Total de registros

2.4 Cabecera alterada.

Esperado: detenerse antes de abrir los resultados en w si la cabecera cambia a id|correo|categoria|detalle. Observado: se mostró “Encabezado incorrecto. Se detuvo el proceso”. Los hashes de las salidas y del historial permanecieron iguales. La captura de referencia muestra el mismo mensaje; la comparación de archivos añade evidencia de que el mensaje correspondió a una detención efectiva.



Prueba 6. Error en cabeceros

3. Conclusiones reflexiones.

El desarrollo de esta práctica evidencia que la preparación de datos requiere un equilibrio constante. Escalar un script a un entorno de producción exige no solo conocer las funciones de manipulación de cadenas, sino anticipar cómo las excepciones afectarán las decisiones del negocio a largo plazo.

3.1. ¿Cómo cambia la distribución si cuentan las etiquetas sin normalizar?

Si las etiquetas se cuentan sin normalizar omitiendo el uso de strip() o la sustitución de guiones, la distribución estadística se fragmentaría. En lugar de obtener tres categorías ("acceso", "pago_en_línea" y "soporte_técnico"), cada variación ortográfica y tipográfica introducida por el usuario se interpretaría como una clase independiente. Esto generaría un aumento artificial de la cardinalidad, nueve etiquetas distintas antes de limpiar y tres después, con conteos normalizados de 9, 9 y 6.

3.2. ¿Qué sesgo podría introducir eliminar todos los rechazos?

Al eliminar permanentemente los 6 registros rechazados (el 20 % de la muestra), se introduce un sesgo de selección. Estaríamos asumiendo que esas solicitudes no tienen valor, cuando en realidad podrían provenir de un segmento específico de usuarios

3.3. ¿Por qué la tasa de aceptación no demuestra exactitud de los datos?

La tasa de aceptación del 80 % (24 registros) demuestra únicamente el cumplimiento de formato (por ejemplo, que el ID inicie con "SOL-" o que el correo contenga un arroba). Sin embargo, la veracidad no se comprueba. Un usuario podría superar estos filtros ingresando un ID perfectamente formateado pero inventado, o un correo incporrecto.

3.4. ¿Qué información se pierde con cada transformación?

Al normalizar los datos se pierden algunos detalles de su escritura original. strip() elimina los espacios extremos; upper() y lower() borran las diferencias entre mayúsculas y minúsculas. Al unificar los separadores de las categorías, ya no podemos distinguir si originalmente se usaron guiones, guiones bajos o espacios. En las descripciones, reducir los espacios consecutivos a uno elimina su cantidad y ubicación, aunque conserva las palabras, mayúsculas y tildes. La inmutabilidad de las cadenas no destruye el texto original: las funciones generan nuevas cadenas. Como conservamos el archivo de entrada, podemos consultar cómo estaba escrito cada registro y revisar las decisiones de limpieza.

3.5. ¿Qué cambiarían si una descripción contuviera “|” o si el archivo superara la memoria disponible?

Si una descripción contuviera |, usaríamos csv.reader con ese delimitador y campos entre comillas para evitar separaciones incorrectas. Para archivos grandes, mantendríamos la lectura por líneas, aunque el conjunto de IDs seguiría creciendo en memoria. Podríamos almacenarlos en SQLite para controlar duplicados. Las pruebas con 30 registros verifican la lógica, pero no demuestran su rendimiento a gran escala.

Declaracion uso de IA.

Se utilizó la IA como apoyo para revisar funciones de validación, persistencia.